

Mechanical Sympathy

How to Make Correct Code an Order of Magnitude Faster

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract. The lock-free mountain was about *correctness* — code that does not break under concurrency. This one is about *speed* — how equally correct code can run an order of magnitude faster. The dividing line is not algorithmic complexity but the **memory wall** between CPU and memory: the CPU is roughly 100× faster than memory and spends most of its time *waiting* on memory, not computing. This report proceeds from five measurable phenomena — **false sharing** (why independent variables drag on each other), **branch prediction** (why a sorted array processes faster), **cache locality** (why sequential access is an order of magnitude faster), **AoS vs SoA** (how data layout sets the effective cache bandwidth), and **the memory wall and prefetch** (why pointer chasing is a performance killer) — measuring each and explaining the mechanism. Throughout, one discipline is stressed: **these micro-architectural penalties are “a few times to a few tens of times,” never a million-fold**; and their mechanisms differ (cache invalidate vs pipeline flush vs cache miss) and must never be conflated. It closes on one thread: HFT's love of preallocated arrays, slot reuse, and no new on the hot path is not folklore — it is because **array addresses are predictable, so prefetch and memory-level parallelism can scale the memory wall; a linked list's pointer chasing leaves memory latency fully exposed, and cannot.**

0. Three Penalties to Keep Apart — the Discipline Running Throughout

Before the five phenomena, one discipline, because it is where error is easiest: **different phenomena have entirely different penalty mechanisms, and they do not transfer.** Apply one phenomenon's vocabulary to another and the analysis is wrong.

Three orthogonal penalties, to be kept strictly apart. **cache invalidate** belongs to *multi-core coherence*: another core wrote the same cache line, so my copy is void — this is false sharing's mechanism, and cannot happen single-threaded. **pipeline flush** belongs to *branch prediction*: the CPU speculated the wrong branch and discards the wrong-path instructions, refilling the pipeline — single-core, nothing to do with cache. **cache miss** belongs to *access locality*: the data is not in cache and must be fetched from memory — single-core, nothing to do with coherence. Their latencies differ too: a cache miss ~100–300 cycles, a pipeline flush ~15–20 cycles, a cache hit ~4 cycles.

The magnitude discipline: a few to a few tens of times, not a million. Every micro-architectural penalty is a **constant-factor** amplification: a miss is tens of times slower than a hit; a flush costs a dozen-odd cycles over a correct prediction. Stacked up they make a program **a few to a few tens of times** slower — *never* a million or ten-million-fold. Any derivation that yields “a million times” has mistaken “one penalty” for “the whole operation redone countless times.” This rule alone blocks a large class of magnitude errors.

1. False Sharing: Why Independent Variables Drag on Each Other

Two threads each hammer a counter, each writing its own, logically fully independent, sharing no variable. Placing the two counters **adjacent** (likely the same 64-byte cache line) versus **padding them onto different lines**, the former is several times slower on multiple cores. The most counter-intuitive case: no sharing, yet mutual drag.

Mechanism: coherence granularity is coarser than the data conflict. The CPU reads/writes memory in whole **cache lines (64 bytes)**, and inter-core coherence (MESI) operates per whole line. Two counters crammed onto one line — even though the two cores write *different bytes* (no logical conflict) — MESI cannot tell them apart; it only sees “both cores want to own the same line.” So core 1’s write must own the line, invalidating core 2’s copy; core 2’s write invalidates core 1’s — **the line ping-pongs between the two cores’ caches**, each bounce transferring the whole line. The sharing is *false* (different bytes), the contention is *real* (same line) — hence false sharing.

Points. First, **single-core never has false sharing**: only one core writes at a time, the line needs no cross-core sync, adjacency is irrelevant. It is a purely *multi-core* phenomenon. Second, the penalty worsens with core count: more cores fighting for one line, worse ping-pong. Third, the fix is trivial — `alignas(64)` or manual padding to push mutually-independent, each-thread-written variables onto different lines; C++17’s `hardware_destructive_interference_size` exists for this.

Do not misstate the mechanism: there is *no lock* here, not “one thread waiting on another’s lock”; it is MESI coherence forcing the two cores to take turns owning the line and repeatedly invalidating each other’s copy — a hardware cache-line fight, not a software lock.

2. Branch Prediction: Why a Sorted Array Processes Faster

An array of random bytes (0-255), a loop doing `if (arr[i] >= 128) sum += arr[i];`. Sorting the array first, then running the same loop, is several times faster than unsorted — though both do identical arithmetic (same comparisons, same number of additions). The only difference is whether the data is sorted.

Mechanism: branch prediction + pipeline flush. The CPU is a deep pipeline; at an if it does not know which way to go but cannot stall — it **speculates**, guessing a direction and pushing the guessed path's instructions into the pipeline early. **Right:** pipeline full, zero waste. **Wrong:** the speculated instructions are all wrong-path and must be **flushed** (discarded, refilled), ~15-20 cycles. *Sorted:* the first half is all <128, the second all ≥128, long runs of identical branch outcomes, the predictor almost always right, few flushes. *Unsorted:* random outcomes, the predictor cannot guess, ~50% of iterations flush.

Do the magnitude arithmetic carefully (this is **addition**, not multiplication): let a no-mispredict iteration cost $\approx C$ cycles (~3-5), a mispredict penalty $\approx P$ cycles (~15-20). Sorted $\approx 1M \times C$; unsorted $\approx 1M \times C + 0.5M \times P$ — unsorted **adds** 0.5M flushes on top of the sorted baseline, it does not multiply each computation by a factor. Measured ratio $\approx 3-6\times$ (the lighter the loop body, the smaller C, the larger the flush share, the higher the ratio). Key insight: **a flush (~18 cycles) is far larger than one computation (~4 cycles)** — precisely because each flush is several times costlier than compute and happens hundreds of thousands of times, it produces the several-fold gap.

A surprise in practice: the compiler auto-branchless-es (CMOV). Compiling the loop directly at -O2, the sorted/unsorted gap may be only $\sim 1.25\times$ — because the compiler turned the if into a **conditional-move (CMOV)**: both paths' results are computed unconditionally, then one is written back by condition — **no jump** → no prediction → no mispredict → no flush. To see the real branch penalty you must write it so the compiler cannot branchless it (e.g. a non-inlinable function call in the branch body); then it measures $\sim 3.2\times$. **The real-world lesson: the branch penalty you imagine is often already erased by the compiler via CMOV;** the gain from hand-written branchless code has often already been taken by the compiler. What still hurts is a branch the compiler *cannot* branchless — one whose body has a call, complex side effects, or a memory write.

3. Cache Locality: Why Sequential Access Is an Order Faster

Traverse an array far larger than L3 (say 256MB, L3 only 18MB), summing all elements. **Sequential** access `arr[0],arr[1],...` versus **random** index access — identical elements, identical additions, only the access *order* differs. Sequential is an order of magnitude faster.

Mechanism: line density + prefetch + memory-level parallelism (MLP). First, cache fetches a whole **64-byte line**. **Sequential:** each line (16 ints) fetched serves 15 more hits, miss rate $\sim 1/16$. **Random:** almost every access hits a new line, miss rate ~ 1 . Second, the **hardware prefetcher** predicts *regular* addresses (arithmetic $\text{base} + i \cdot 4$) and fetches upcoming lines early — sequential misses are largely hidden; random addresses are irregular, prefetch fails. Third, **MLP:** sequential addresses are all computable in advance, so the CPU issues many misses at once and *overlaps* their latency; random-index access can still compute addresses (unlike pointer chasing), so MLP partially works while prefetch does not. Net measured ratio typically **5-30 $\times$** .

Get the magnitude right. A cache miss to DRAM is ~ 100 -300 cycles, an L1 hit ~ 4 cycles, so **single miss/hit $\approx 75\times$** . But the **total-time ratio is not 75**: the sequential version is not pure hits (it carries a $1/16$ amortized miss + prefetch help), so the denominator is not 4; the random version has MLP overlap, so the numerator is not that bad either. Hence total ratios land at 5-30, not 75. Deliberately constructing “every access jumps to a far new page” further stacks the TLB miss below, pushing the ratio toward the high end.

A second cache: the TLB (address-translation cache) — don’t conflate it with the data cache. Programs use **virtual addresses**; data lives at **physical addresses**; translation goes through a **page table** (in memory). Every access needs a translation — too slow to walk the page table each time, so the CPU caches recent “virtual page $\rightarrow$ physical page” mappings in the **TLB**. A **TLB miss** = the translation is not in the TLB, so it walks the page table *in memory* (tens to hundreds of cycles) — **still in memory, not disk**. Disk is only touched by a **page fault**: the data is not in memory at all and must be paged in from disk, $\sim$ a million cycles, tens of thousands of times slower. The three-tier ladder: L1 ~ 4 cycles $\rightarrow$ DRAM/TLB-walk $\sim$ hundreds $\rightarrow$ disk $\sim$ millions. Sequential access hits the TLB (one 4KB page = 1024 ints); large-stride random access misses the TLB frequently, on top of the data miss. The TLB is **one-per-core hardware**, shared by processes, distinguished by process tags (ASID/PCID) — one reason HFT pins and isolates cores, to avoid context switches polluting TLB/cache. HFT also locks and pre-faults memory so the run never page-faults.

4. AoS vs SoA: How Layout Sets Effective Cache Bandwidth

A million particles, each with position (x,y,z), velocity (vx,vy,vz), and a pile of other fields (mass, color, temp...), the struct assumed 64 bytes. A hot loop **updates only position**: `x += vx; y += vy; z += vz;`, touching 6 fields, not mass/color/temp. Two layouts:

- **AoS (array of structs):** Particle `arr[N]` — “all fields of one particle adjacent.”
- **SoA (struct of arrays):** `float x[N], ..., vx[N], ...` — “the same field of all particles adjacent.”

Mechanism: the effective data density of a cache line. Updating only position: **AoS** fetches a whole 64-byte line to get particle 0’s `x` = all of particle 0, but uses only 24 of those bytes (the 6 position/velocity fields), **wasting the other 40** (mass/color/temp) — each line just 38% effective. **SoA** touches `x[]`: one line = 16 particles’ `x`, **all usable**; `vx[]` likewise — each line 100% effective. So for the same position work SoA moves ~38% of AoS’s memory traffic, measured **2-4× faster** (theoretical bound $\approx 100/38 \approx 2.6$, with prefetch/SIMD to 2-4). The principle: **make everything in a cache line be data you are about to use.**

SoA is not unconditionally better; it has a **hidden prerequisite**: the cache must be large enough to hold the current line of every array the hot loop needs (here 6). Normal caches (L1 alone has hundreds of lines) satisfy this easily, so SoA usually wins. But squeeze the cache to an extreme (say one line) and SoA’s 6 fields on 6 different lines cannot fit, **thrashing** — now AoS wins (one particle’s fields on one line, one line serves one particle). The rule: **batch-process a few fields → SoA; per-object access to all fields → AoS; in between, AoSoA** (small AoS blocks within a line, SoA across blocks). Columnar storage in ML/numerics is everywhere the SoA idea — vectorizing over a feature column puts all its values adjacent, saturating cache and SIMD.

5. The Memory Wall and Prefetch: Why Pointer Chasing Kills

One number first: the CPU does several arithmetic ops per cycle, while a cache miss to DRAM costs ~100-300 cycles — **the CPU is ~100× faster than memory**, a gulf called the **memory wall**. The CPU spends most of its time *waiting* on memory. Traverse a linked list whose nodes are scattered across the heap (pointer chasing), doing a little compute per node:

```
Node* p = head;
while (p) {
    sum += compute(p->value); // a little computation
    p = p->next;              // jump to the next node (random address)
}
```

Mechanism: a serial dependency chain leaves latency unhideable. The bottleneck is $p = p \rightarrow \text{next}$'s **memory access**, not compute's arithmetic — compute is a few cycles, but each jump to a new node waits a ~ 200 -cycle miss, the CPU mostly **stalled**, ALU idle. **Prefetch cannot save it**, because of a chicken-and-egg serial dependency: to prefetch a farther node you need its address; but the "node-after-next" address is locked inside the "next node's contents," which have **not been fetched yet**. You hold only the *immediate* next address (it shares the current node's line with value, already hit), but hiding a 200-cycle latency needs looking many steps ahead — a list can only **unlock addresses one step at a time**, each step serially waiting a miss.

Why arrays scale the wall: array addresses are **computable in advance** ($\text{base} + i * 4$), not dependent on "the contents of the last access." So the CPU can **prefetch** (fetch upcoming lines early) and use **MLP** (issue many misses at once, overlapping latency). Both *hide* memory latency. **Pointer chasing loses both**: addresses uncomputable (prefetch dead), next address depends on the prior contents (MLP dead). So list traversal is $\text{latency} \times \text{node_count}$ of pure serial waiting; an array is $\text{latency} \div \text{parallelism}$ of overlapped waiting — the **root reason** a list is an order of magnitude slower than an array, not merely "scattered nodes, high miss rate," but **a serial dependency leaving latency fully exposed, unhideable**.

Why HFT clings to preallocated arrays — all threads converge here. HFT's preference for **preallocated arrays, slot reuse, no new on the hot path** has two layers, now joined. One (reclamation): array slots are reused, never deleted, sidestepping the whole safe-reclamation problem and immune to ABA for free (monotone integer indices, not reused pointers). Two (memory wall, this report): array addresses are predictable $\rightarrow$ prefetch/MLP scale the wall; a list's pointer chasing leaves latency fully exposed and cannot. **The two mountains, "correctness" and "speed," point to one engineering conclusion: if it can be contiguous, predictable, and reused, do not dynamically allocate and do not chase pointers.**